

UNIVERSIDADE CATÓLICA DE BRASÍLIA
CIÊNCIA DA COMPUTAÇÃO

JHONY MORAES DA SILVA
JOÃO VÍTOR FERREIRA CALADO
KAUÃ LACERDA DE LIMA
RAFAEL DE SOUSA SILVA

DOCUMENTO DE PLANO DE TESTES
SISTEMA GERENCIADOR DE LOJA - (SGL)

BRASÍLIA, DF
2026

JHONY MORAES DA SILVA
JOÃO VÍTOR FERREIRA CALADO
KAUÃ LACERDA DE LIMA
RAFAEL DE SOUSA SILVA

DOCUMENTO DE PLANO DE TESTES
SISTEMA GERENCIADOR DE LOJA - (SGL)

Documento apresentado a Universidade Católica de Brasília, campus Taguatinga, como requisito para aprovação bimestral parcial no curso de Teste de Software.

Orientador: Prof. Samuel Novais de Moura Júnior

BRASÍLIA, DF
2026

RESUMO

O presente trabalho aborda o planejamento, a estruturação e a aplicação de estratégias de teste de software voltadas para o desenvolvimento do Sistema Gerenciador de Loja (SGL). A pesquisa foi desenvolvida a partir de uma abordagem metodológica exploratória, descritiva e explicativa, combinando fundamentação teórica sobre engenharia de qualidade de software com a aplicação prática em um sistema desenvolvido em Java, contemplando arquitetura em camadas, banco de dados relacional via JDBC e interface de linha de comando (CLI). O objetivo geral deste estudo consiste em planejar e documentar cenários de testes unitários, de integração e ponta a ponta (E2E) capazes de assegurar a robustez das regras de negócio, a integridade transacional dos dados e a usabilidade dos fluxos do sistema. De acordo com o estudo bibliográfico e prático desenvolvido, foi possível evidenciar a importância da automação de testes com ferramentas modernas na detecção precoce de falhas estruturais, mitigando riscos de corrupção de dados e inconsistências em operações críticas, como o controle de estoque e o processamento de pedidos. Para o embasamento teórico, utilizou-se literatura clássica e contemporânea voltada para a garantia de qualidade de software e metodologias de teste estrutural e funcional. Por fim, a pesquisa constatou que a aplicação sistemática de um plano de testes bem estruturado otimiza o ciclo de vida do desenvolvimento, elevando consideravelmente a confiabilidade da aplicação comercial SGL.

Palavras-chave: Teste de Software; JUnit 5; Garantia de Qualidade; Sistema Gerenciador de Loja; Engenharia de Requisitos.

SUMÁRIO

1 INTRODUÇÃO	5
2 PLANO DE TESTE.....	6
2.1 INTRODUÇÃO E OBJETIVO DOS TESTES	6
2.2 ESCOPO DO TESTE	6
2.2.1 O que será testado (In Scope):.....	6
2.2.2 O que NÃO será testado (Out of Scope):.....	7
2.4 ESTRATÉGIA DE TESTES.....	7
2.4.1 Testes Unitários	7
2.4.2 Testes de Integração e Camada de Dados (DAOs)	8
2.4.3 Testes de Carga e Desempenho (Apache JMeter).....	8
2.4.4 Testes End-to-End (E2E) e de Interface (CLI).....	8
2.5 PAPÉIS E RESPONSABILIDADES (EXEMPLO PARA O GRUPO)	8
3 CONSIDERAÇÕES FINAIS.....	10
REFERÊNCIAS	11

1 INTRODUÇÃO

A evolução e a complexidade dos sistemas de software comerciais exigem processos rigorosos de garantia de qualidade. No contexto de sistemas gerenciadores de estabelecimentos comerciais, a integridade das informações — como controle de estoque, fluxo de caixa, cadastros de clientes e histórico de pedidos — é um fator crítico para a sustentabilidade operacional do negócio. Falhas em regras de negócio ou inconsistências em transações de banco de dados podem gerar prejuízos financeiros severos e perda de credibilidade.

Este documento apresenta o Plano de Testes oficial para o Sistema Gerenciador de Loja (SGL). O SGL foi estruturado utilizando uma arquitetura robusta em Java, contemplando classes de modelo com validações estritas de domínio, persistência orientada a objetos via JDBC através de DAOs (Data Access Objects) dedicados, além de uma interface de linha de comando (MainCLI) para a interação do operador de caixa e administradores.

O propósito deste plano é mapear os principais riscos, delimitar o escopo de verificação e validação, definir as ferramentas adequadas (com foco especial em JUnit 5 para testes automatizados unitários e de integração) e estabelecer diretrizes claras para a execução de casos de teste que cubram desde unidades isoladas de código até fluxos completos de ponta a ponta (E2E). As seções seguintes detalham o escopo, as estratégias metodológicas e os cenários práticos mapeados para assegurar a entrega de um software estável e confiável.

2 PLANO DE TESTE

2.1 INTRODUÇÃO E OBJETIVO DOS TESTES

O presente capítulo detalha o planejamento, o escopo operacional e a estratégia metodológica para a execução das atividades de teste no Sistema Gerenciador de Loja (SGL). O objetivo principal desta etapa é estabelecer critérios rigorosos de verificação e validação que assegurem a estabilidade, a integridade estrutural e a conformidade funcional do software perante os requisitos especificados.

O processo contempla uma pirâmide de testes bem definida, abrangendo testes unitários voltados para as validações de domínio nas classes de modelo, testes de integração e persistência focados na camada de acesso a dados (DAOs) via JDBC e transações atômicas, testes de desempenho e carga para avaliar o comportamento do sistema sob estresse utilizando o Apache JMeter, e testes de ponta a ponta (E2E) simulando o comportamento interativo do usuário na interface de linha de comando (MainCLI).

2.2 ESCOPO DO TESTE

2.2.1 O que será testado (In Scope):

Camada de Modelos e Regras de Negócio (Models): Validação estrita de restrições de domínio, garantindo que atributos críticos (como preços unitários, quantidades em estoque, formatação de documentos cadastrais e status permitidos para pedidos e pagamentos) rejeitem dados inconsistentes por meio do lançamento adequado de exceções.

Camada de Acesso a Dados e Persistência (DAOs): Verificação completa das operações de CRUD (Create, Read, Update, Delete) para todas as entidades principais do sistema: Clientes, Categorias, Produtos, Fornecedores e Vendedores, assegurando o correto mapeamento relacional via JDBC.

Transações Complexas e Integridade Atômica: Validação de fluxos transacionais complexos, tais como o método `inserirPedidoCompleto`, `atualizarPedidoCompleto` (com reversão lógica de estoque) e `deletarPedidoCompleto` (com reposição automática de itens ao inventário), controlando explicitamente o gerenciamento de transações com `setAutoCommit(false)`, `commit()` e `rollback()`.

Interface de Linha de Comando (MainCLI): Validação dos fluxos de navegação em menus e submenus, autenticação de operadores e tratamento de exceções decorrentes de entradas textuais inválidas inseridas pelo usuário no terminal.

Desempenho e Concorrência (Carga): Avaliação do comportamento da camada de persistência e conexões com o banco de dados sob cenários de requisições concorrentes simuladas por ferramentas de teste de carga.

2.2.2 O que NÃO será testado (Out of Scope):

- Desempenho e latência de infraestruturas de rede externa, provedores de hospedagem em nuvem ou instâncias físicas de servidores terceirizados.
- Mecanismos avançados de criptografia de senhas em infraestrutura de rede, visto que o escopo de segurança atual restringe-se à validação lógica local da aplicação CLI.
- Testes de compatibilidade gráfica ou visual, uma vez que o sistema opera predominantemente em ambiente de terminal (CLI), desconsiderando interfaces gráficas complexas de desktop nesta fase.

2.3 FERRAMENTAS UTILIZADAS

Para a execução e automação da qualidade no SGL, foram adotadas ferramentas padrão de mercado recomendadas para o ecossistema Java:

JUnit 5: Utilizado como framework principal para o desenvolvimento e execução de testes unitários isolados e testes de integração orientados a banco de dados. Permite o uso de anotações avançadas (`@Test`, `@BeforeEach`, `@ParameterizedTest`, `@DisplayName`) para estruturar cenários de sucesso e falha de forma limpa e organizada.

Apache JMeter: Empregado nos testes de desempenho, carga e estresse. Permite simular múltiplos usuários executando consultas e operações simultâneas na base de dados para mensurar o tempo de resposta e a estabilidade do SGL sob concorrência.

Drivers JDBC e Bancos de Dados de Teste: Conexões dedicadas a ambientes de homologação/teste para assegurar que a execução dos scripts de teste não corrompa os dados de produção da loja.

2.4 ESTRATÉGIA DE TESTES

2.4.1 Testes Unitários

Os testes unitários têm como foco isolar as classes de domínio e modelo (Cliente, Produto, Pagamento, Fornecedor, Categoria, entre outras) para validar suas regras de negócio de forma independente de banco de dados ou interfaces externas. O propósito é garantir que o sistema dispare exceções do tipo `IllegalArgumentException` ou equivalentes sempre que houver tentativa de inserção de valores inválidos (por exemplo, cadastro de um produto com preço

negativo, strings vazias em campos obrigatórios ou quantidades de estoque inválidas). Esses testes asseguram a consistência imediata do estado dos objetos logo em sua instanciação.

2.4.2 Testes de Integração e Camada de Dados (DAOs)

Esta estratégia visa validar a comunicação real e integrada entre as classes DAO e o banco de dados relacional. Serão testadas as operações de persistência e recuperação de registros para garantir que o mapeamento SQL executado via JDBC esteja correto e livre de falhas de sintaxe ou de mapeamento de tipos. Além disso, o foco central recai sobre os cenários transacionais: testa-se explicitamente se o uso de transações (rollback em caso de exceções e commit em caso de sucesso) protege o banco de dados contra estados corrompidos, como ocorreria caso uma venda fosse registrada sem saldo suficiente em estoque.

2.4.3 Testes de Carga e Desempenho (Apache JMeter)

Utilizando o Apache JMeter, a estratégia de testes de carga simula a concorrência de múltiplos usuários acessando rotinas de consulta de produtos, listagem de clientes e inserção de pedidos de forma simultânea. O objetivo é monitorar o comportamento do pool de conexões JDBC e identificar possíveis gargalos de desempenho ou travamentos na camada de persistência sob estresse operacional, garantindo que o SGL suporte picos de acesso típicos de um ambiente comercial real.

2.4.4 Testes End-to-End (E2E) e de Interface (CLI)

Os testes de ponta a ponta simulam a jornada completa do operador ou administrador de loja utilizando a interface de linha de comando (MainCLI). O fluxo compreende desde a inicialização do sistema, autenticação de credenciais de vendedor, navegação pelos menus de gerenciamento de produtos e fornecedores, até a execução completa de uma venda (criação do pedido, associação de itens, baixa no estoque e registro do pagamento). O propósito é validar a coesão entre todas as camadas da aplicação, desde a interação via terminal até a persistência definitiva das informações no banco de dados.

2.5 PAPÉIS E RESPONSABILIDADES (EXEMPLO PARA O GRUPO)

Para viabilizar a execução eficiente do plano de testes no contexto da equipe de desenvolvimento, as atribuições foram distribuídas da seguinte forma:

Desenvolvedor / Analista de Testes Unitários: Responsável por implementar, manter e executar os casos de teste em JUnit 5 voltados para a validação das classes de modelo e regras de domínio puras.

Responsável por Banco de Dados e DAOs: Encarregado de estruturar a massa de dados inicial (fixtures), configurar o ambiente de banco de dados de teste e validar a integridade dos testes de integração e transações complexas (Commit/Rollback).

Responsável por Testes de Carga (JMeter) e E2E: Encarregado de configurar os planos de teste de concorrência no JMeter, simular os fluxos de ponta a ponta na interface CLI e consolidar as métricas de desempenho e evidências de execução do sistema.

3 CONSIDERAÇÕES FINAIS

O desenvolvimento e a aplicação deste Plano de Testes para o Sistema Gerenciador de Loja (SGL) demonstraram a relevância de estruturar a qualidade de software desde as fases iniciais do projeto. Por meio da definição clara de escopo, da divisão estratégica entre testes unitários, de integração e de ponta a ponta, e da escolha de ferramentas adequadas como o JUnit 5, foi possível estabelecer um processo metodológico rigoroso.

A verificação sistemática das classes de modelo garantiu que nenhuma violação de domínio passe despercebida pela aplicação. Da mesma forma, os testes voltados para a camada de acesso a dados (DAOs) e transações asseguraram que o controle de estoque e o fluxo de pedidos ocorram de maneira atômica e segura, evitando inconsistências no banco de dados relacional. Em suma, o trabalho cumpriu o objetivo proposto de elevar o padrão de confiabilidade do SGL, evidenciando que a engenharia de testes é um pilar indispensável para o desenvolvimento de software profissional e resiliente.

REFERÊNCIAS

PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson Education do Brasil, 2019.